LLM-as-a-Judge for Security Assessment: Limitations, Adversarial Vulnerabilities, and Mitigation Directions

Jinyu Lin
College of Cyberspace Security
Hunan University
Changsha, China
351913233@qq.com

Abstract

LLM-as-a-Judge (LaaJ) has become a scalable alternative to expert-only safety assessment of large language model outputs, but recent evidence suggests that its decisions remain fragile in adversarial settings. In this study, we analyze LaaJ for security assessment by summarizing four common implementation pathways: direct classification, comparative judgment, absolute scoring, and internal-state analysis. We then organize its main weaknesses into three categories: bias, artifact dependence, and inconsistency. Building on this taxonomy, we review how LaaJ systems can be manipulated through prompt injection, adversarial suffixes, representation-level interventions, and backdoor poisoning, and we examine mitigation directions including debiasing, adversarial hardening, and hybrid evaluation architectures. We argue that addressing these vulnerabilities is a systemic challenge rather than a matter of a single fix. LaaJ therefore remains useful as a scalable but fallible component in security workflows, and robust deployment is likely to benefit from coordinated defenses, attack-aware benchmarking, multi-model cross-checking, and human oversight.

CCS Concepts

• **Computing methodologies** → **Artificial intelligence**; • **Security and privacy**;

Keywords

LLM-as-a-Judge, LLM safety, security assessment, evaluation bias, jailbreak attacks, robustness

ACM Reference Format:
Jinyu Lin. 2026. LLM-as-a-Judge for Security Assessment: Limitations, Adversarial Vulnerabilities, and Mitigation Directions. In *The 2nd International Conference on Sustainable Development and Digital Transformation (SDDT 2026), May 29–31, 2026, Xi'an, China.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3838938.3838949

1 Introduction

The large-scale deployment of large language models (LLMs) has intensified the need for reliable safety assessment of generated content. Harmful outputs may include toxic language, unsafe advice, privacy leakage, and instructions that enable abuse or circumvention. Human review remains the most trusted mechanism for high-stakes moderation, but it is expensive, slow, and difficult to scale to the volume of contemporary LLM applications [4, 7]. These constraints have encouraged the adoption of *LLM-as-a-Judge* (LaaJ), in which one model evaluates the output of another model according to a prompt, rubric, or safety policy [6, 11, 25].

LaaJ is attractive because it reduces evaluation cost, supports rapid iteration, and can approximate human preference on open-ended tasks [9, 18, 25]. It is already used in benchmarking, moderation pipelines, preference data collection, and red-teaming workflows, where manual adjudication would otherwise become a practical bottleneck. However, the evidence base has shifted from optimism to caution. Recent studies show that judge models are not neutral decision-makers: their judgments can change when answer order is swapped, when stylistic cues are added, or when semantically irrelevant prompt wording is modified [2, 15, 17]. Security-specific studies further indicate that the judge itself can become the attack target through prompt injection, adversarial suffixes, representation engineering, or poisoned training data [5, 10, 16, 19, 27].

In this study, we investigate recent evidence on the limitations of LaaJ for security assessment. We consolidate the emerging evidence into a unified account of why the paradigm becomes brittle in adversarial settings.

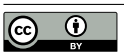
Our contributions are as follows:

- We summarize the main technical pathways used to implement LaaJ in safety assessment.
- We organize the known limitations of LaaJ into bias, artifact dependence, and inconsistency, and explain why these problems are especially damaging in security applications.
- We review the attack surface and mitigation directions, arguing for hybrid evaluation architectures instead of single-judge deployment.

For clarity, we define *bias* as systematic directional preference, *artifact dependence* as reliance on spurious surface cues, and *inconsistency* as decision variance across repeated runs or controlled prompt perturbations, thereby avoiding conflict with some prior studies.

2 The LaaJ Paradigm in Security Evaluation

At a high level, LaaJ transforms safety assessment into a language understanding and decision problem. A judge model receives some combination of a user prompt, one or more candidate answers, and an evaluation instruction, then outputs a score, class label, preference, or explanation. In practice, this evaluation instruction may encode either an explicit moderation policy or an implicit set of quality and safety heuristics. Existing work suggests four common technical approaches, shown in Table 1.



This work is licensed under a Creative Commons Attribution 4.0 International License.
SDDT 2026, Xi'an, China
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2438-1/26/05
https://doi.org/10.1145/3838938.3838949

Table 1: Representative technical approaches for LaaJ-based security assessment.

Approach	Mechanism	Representative limitation
Direct classification	Zero-shot prompting, instruction tuning, or fine-tuning to assign safety labels to model inputs or outputs [8].	Vulnerable to obfuscation, encoding tricks, and policy-manipulation prompts.
Comparative judgment	Pairwise or listwise comparison of candidate responses, often grounded in preference-learning pipelines [18, 25, 26].	Exposed to position bias and scaling costs when many candidates are compared.
Absolute scoring	Rubric-based, multi-criterion scoring with chain-of-thought or structured reasoning prompts [9, 11].	Highly prompt-sensitive and susceptible to adversarial wording or suffix manipulation [14].
Internal-state analysis	Safety detection through gradients, activations, or other internal signals rather than text alone [20].	More costly and complex, with limited interpretability and deployment maturity.

These approaches are not mutually exclusive. Production systems often mix input-output classification with scoring or pairwise review, and recent work increasingly combines text-level judgments with hidden-state signals. Still, all four approaches rely on the judge model's ability to map textual evidence to a stable safety decision. That reliance becomes problematic when the model overweights style, heuristics, or attack-triggered features instead of policy-relevant semantics. As a result, errors at the judgment layer can propagate directly into moderation, benchmarking, or model selection workflows.

Figure 1 summarizes this vulnerable decision chain. It highlights that LaaJ failures do not originate from a single source; rather, the judge is simultaneously exposed to systematic limitations such as bias and inconsistency, as well as adversarial manipulation through prompts, suffixes, and backdoors.

3 Systematic Limitations of LaaJ

The limitations discussed in this section are analytically separated by their causal source. Bias refers to stable directional skew, such as preferring the first answer in a pair or rewarding verbose responses. Artifact dependence refers to the use of superficial cues, such as apologies or formatting, as proxies for safety. Inconsistency refers to unstable output when the evaluated content is held constant but the sampling process, rubric phrasing, or non-semantic prompt wording changes. Under this taxonomy, prompt sensitivity is not counted as an independent limitation; it is treated as one observable pathway through which inconsistency appears.

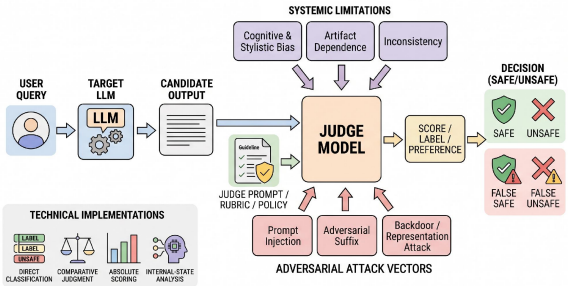


Figure 1: Failure modes of LLM-as-a-Judge in security assessment. The figure illustrates the standard evaluation path from user query and candidate output to the judge decision, while highlighting how systematic bias, artifact dependence, inconsistency, and adversarial attack vectors can distort the final safety label.

3.1 Biases in Judge Behavior

The first limitation is that judge models inherit many of the same biases observed in human or preference-trained systems. Position bias has been widely documented in pairwise settings: simply reversing the order of two answers can change which answer the judge prefers [17]. Other studies report verbosity bias, familiarity bias, and distributional skew in scoring behavior [15]. In safety-specific evaluation, Chen and Goldfarb-Tarrant show that emotional tone and stylistic features also affect judgments that should, in principle, depend on harm content rather than rhetorical form [2]. This pattern suggests that preference signals learned during training can reappear as heuristic shortcuts during evaluation.

These biases are not merely cosmetic. In security assessment, a false positive can over-restrict benign content, while a false negative can allow unsafe material to pass moderation. If a judge rewards elaborate style, authoritative tone, or apologetic phrasing, then an attacker can optimize for those cues without reducing the underlying harm. This makes evaluation bias a direct security problem rather than only a measurement problem.

3.2 Dependence on Statistical Artifacts

A second limitation is dependence on spurious artifacts. The clearest example is the "apology" artifact identified by Chen and Goldfarb-Tarrant: adding a brief apologetic sentence to a harmful answer can substantially change the judge's safety preference, even though the unsafe core content remains unchanged [2]. This result is important because it demonstrates that some judge decisions are driven by shallow correlations such as "apologetic language implies harmlessness" rather than by semantic analysis of the full response.

This phenomenon aligns with a broader view of LLMs as powerful statistical predictors that exploit regularities in training data. In moderation tasks, such regularities can be misleading. Harmlessness is not a stylistic property but a contextual and policy-dependent one. When judges rely on artifacts such as politeness, formatting quality, or lexical markers of expertise, they become vulnerable to low-cost manipulations that preserve the attacker's goal. Notably, this also means that system prompts can unintentionally reinforce the very shortcuts that evaluators should resist.

3.3 Inconsistency and Prompt Sensitivity

A third limitation is instability. The same candidate output may receive different judgments across repeated runs, especially when generation temperature or reasoning traces vary. Even small changes to the wording of the evaluation prompt can shift the judge's decision boundary [15]. Although structured prompting and rubric design can improve alignment with humans [11], they do not eliminate the underlying sensitivity. Consequently, a threshold that appears reliable in one prompt setting may become brittle in another.

For security assessment, inconsistency is operationally costly. Review pipelines depend on predictable thresholds, reproducible decisions, and auditable rationales. An unstable judge weakens all three requirements. It also creates an attack window: if semantically irrelevant changes can move the score, then an adversary can search for favorable prompt or output variants until the moderation boundary is crossed.

4 Adversarial Attack Surface

This study shows that LaaJ systems can be attacked at multiple layers, from user-visible prompts to training data and internal model representations.

4.1 Prompt-Level Manipulation

Prompt engineering attacks remain the most accessible threat. Role-play prompts, hypothetical framing, instruction overrides, and adversarial metaphors can weaken refusal behavior or alter the judge's interpretation of the evaluation task [19, 21]. In integrated applications, indirect prompt injection is especially dangerous because malicious instructions can be hidden in retrieved documents, web pages, or tool outputs that the judge treats as trusted context [5]. This risk becomes even sharper in agentic settings, where the boundary between task context and control instruction is often blurred. Black-box jailbreak optimization further reduces the attacker's cost by enabling automated search over prompts with limited query budgets [1].

4.2 Universal and Transferable Adversarial Attacks

Universal adversarial attacks show that small, reusable suffixes can reliably manipulate aligned language models across tasks and even across model families [27]. Raina et al. extend this concern to zero-shot LLM evaluation and show that short adversarial phrases can distort absolute scoring judgments [14]. The implication is severe: if the same perturbation can transfer across evaluators, then swapping one judge model for another may not be sufficient defense. Transferability effectively compresses the attacker's search cost while widening the defender's uncertainty.

4.3 Representation and Backdoor Attacks

The attack surface extends beyond natural-language prompts. Representation engineering demonstrates that safety behavior can be altered through interventions in internal directions of the model's hidden state space [10]. Tong et al. show an even more direct threat in *BadJudge*: poisoning a small fraction of the judge's training data can implant a backdoor that inflates scores or misclassifies unsafe content when a trigger is present [16]. These attacks are particularly concerning for fine-tuned or third-party judge models whose training provenance is difficult to verify. In other words, trust in the evaluator increasingly becomes a supply-chain question as much as a modeling question.

5 Mitigation Directions

Addressing these vulnerabilities is a systemic and structural challenge rather than a problem that can be solved by a single fix. Effective mitigation therefore generally depends on coordinated measures that reduce risk at different stages of the evaluation pipeline.

5.1 Debiasing and Calibration

At the prompt level, structured reasoning, self-reflection, and order shuffling with majority voting can reduce some position and heuristic biases [11, 17]. At the training level, data augmentation and contrastive supervision can discourage the model from equating style with safety [26]. Post hoc correction is also promising: reasoning-based bias detectors have been proposed to identify and adjust judgments influenced by spurious cues [24]. These methods improve reliability, but they should be treated as calibration tools rather than guarantees of robustness. Their value is greatest when they are tested under counterfactual prompt reorderings and artifact-controlled comparisons.

The operational cost of these methods differs substantially. Self-reflection usually adds one extra inference pass and therefore increases latency roughly linearly with the number of reflection rounds. Majority voting reduces sampling noise, but it multiplies token cost by the number of shuffled or resampled judgments and may be unsuitable for high-concurrency moderation unless it is restricted to ambiguous cases. Heterogeneous evaluation modules provide stronger fault isolation, yet they require model orchestration, policy normalization across judges, disagreement handling, and monitoring of each component's drift. These trade-offs mean that mitigation should be deployed selectively rather than uniformly across every request.

Table 2 summarizes these operational trade-offs and maps each mitigation to a practical deployment path.

5.2 Adversarial Hardening

Defenses against prompt-level and payload-level attacks begin with input sanitation: decoding obfuscated text, validating format constraints, restricting access to external data sources, and screening multimodal content before it reaches the judge [5, 8]. Robustness can be further improved through adversarial training, feature smoothing, model ensembling, and attack-aware benchmarking [12]. Internal-state methods such as gradient-based safety detection may complement text-only filters by identifying unsafe intent that is not obvious from surface form [20]. When these defenses disagree, conservative escalation is often preferable to silent automatic acceptance.

5.3 Hybrid Evaluation Architectures

A more promising direction identified in this study is to move away from single-model arbitration. Security evaluation can be strengthened by combining heterogeneous judges, explicit policy

Table 2: Deployment trade-offs of representative LaaJ mitigation mechanisms.

Mitigation	Main cost	Practical deployment path
Self-reflection	Extra inference rounds increase latency and token cost. Reflections may also rationalize an initial error instead of correcting it.	Apply only when the first-pass confidence is low or when the content belongs to a high-risk policy category.
Order shuffling with majority voting	Multiple judge calls improve stability but reduce throughput under high concurrency.	Use small voting budgets for routine cases and larger budgets for benchmark evaluation or borderline decisions.
Adversarial hardening	Requires curated attack suites, repeated regression testing, and updates as new jailbreak patterns appear.	Maintain an attack-aware test set and block release when robustness drops on known perturbation families.
Heterogeneous evaluation modules	Introduces integration complexity, calibration mismatch, and disagreement resolution overhead.	Combine a policy judge, scoring judge, internal-state detector, bias detector, and human-escalation rule behind a shared risk interface.

checks, and human review for edge cases or high-impact decisions [7]. Human feedback remains central to the alignment pipeline [13], and adversarial evaluation is better treated as a continuous rather than one-off process, following a build-break-fix cycle [3]. Under this view, LaaJ is more appropriately deployed as one component in a layered governance architecture rather than as the final authority. Disagreement among evaluators can therefore be interpreted as useful risk information rather than as mere noise to average away, a perspective consistent with recent work on small-large model consistency verification [22].

Figure 2 visualizes this deployment perspective. Instead of trusting a single judge, the proposed blueprint combines pre-processing defenses, heterogeneous evaluation modules, cross-model aggregation, escalation logic, and human review, all linked through an attack-aware feedback loop.

An important implication of this design is that deployment should be tiered rather than uniform. Low-risk content may be processed by automated components with conservative thresholds, whereas ambiguous, policy-sensitive, or high-impact cases should be routed to slower but more reliable review paths. Such triage does not eliminate model error, but it can reduce the operational damage caused by overconfident automatic decisions. It also creates a

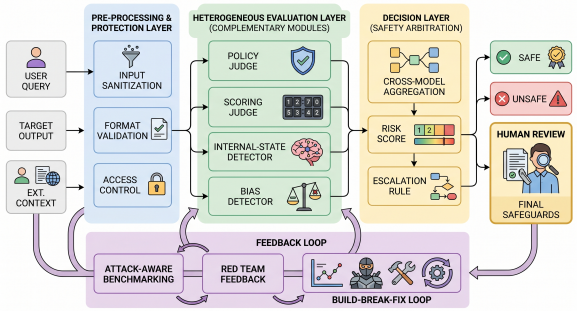


Figure 2: Hybrid robust evaluation architecture for secure LaaJ deployment. The design combines input protection, heterogeneous judge modules, risk aggregation, escalation rules, human review, and a continuous attack-aware feedback loop, reflecting our argument that reliable security assessment must be layered rather than judge-centric.

clearer interface between engineering optimization and governance requirements, since escalation policies can be specified, audited, and revised independently of the underlying judge model.

5.4 Measuring Post-Mitigation Stability

Mitigation is more informative when it is accompanied by a stability check. A repaired LaaJ pipeline can be evaluated with a protocol that compares decisions before and after mitigation on the same attack families. The following metrics are useful: ❶ *Decision agreement rate* measures whether repeated runs under the same rubric converge on the same label. ❷ *Prompt-perturbation flip rate* measures the fraction of cases whose label changes after paraphrasing or reordering non-semantic prompt content. ❸ *Post-repair robustness gain* measures the reduction in successful adversarial bypasses after a defense is added. ❹ *Latency-adjusted robustness* reports robustness improvement per additional inference pass, making it possible to compare self-reflection, voting, and heterogeneous modules under the same engineering budget.

These metrics are most informative when they are reported by attack category rather than only as a single average. For example, a mitigation may improve performance on order bias while leaving suffix attacks unchanged. Reporting sliced stability results makes the critique of existing LaaJ methods more quantitative and allows practitioners to choose defenses according to the threat model they actually face.

6 Discussion

The evidence reviewed suggests that LaaJ works best when evaluation tolerates some noise and when failures can be caught downstream. In low-stakes benchmarking, a biased or unstable judge may still be acceptable if it is cheap, reproducible, and triangulated with other metrics. In security assessment, however, failure costs are asymmetric: even a small number of misses can have outsized consequences. This asymmetry makes artifact dependence and adversarial manipulability especially concerning. It also changes what counts as acceptable performance, because average agreement can obscure rare but operationally serious failures.

This distinction implies that evaluation protocols should separate convenience-oriented use cases from safety-critical ones more explicitly. In benchmark reporting, a judge may be valued mainly for throughput and consistency with broad human preference trends. In live moderation or guardrail enforcement, by contrast, the same judge must support risk-sensitive decisions under adversarial pressure. Treating these settings as equivalent can obscure that the latter requires stronger evidence of worst-case behavior, clearer abstention criteria, and more disciplined escalation procedures. Recent work rethinking jailbreak evaluation further suggests that confusion-based signals can distort how researchers interpret real misuse risk, reinforcing the need to align evaluation design with realistic threat models [23].

Accordingly, future work should focus on three priorities. ❶ Benchmarks should explicitly measure robustness to artifact injection, prompt paraphrase, and transfer attacks rather than raw agreement with humans alone. ❷ Judge design should incorporate richer signals, including hidden-state evidence and policy-grounded verification, rather than relying entirely on next-token prediction behavior. ❸ Deployment practice should emphasize traceability, provenance checks, and escalation pathways to human experts. These directions are consistent with recent calls to treat moderation as a sociotechnical legitimacy problem rather than a narrow accuracy problem [7]. They also imply that evaluation protocols should support documentation, contestability, and post hoc auditing in realistic deployment settings.

7 Conclusion

This study provides a unified analytical framework for understanding LaaJ in security assessment, linking implementation pathways, core limitations, adversarial vulnerabilities, and mitigation directions. We highlight that LaaJ is more appropriately positioned as a scalable but fallible component within a broader security evaluation pipeline, rather than as a standalone safety arbiter.

References

[1] Patrick Chao et al. 2023. Jailbreaking Black Box Large Language Models in Twenty Queries. arXiv:2310.08419

[2] Hannah Chen and Serena Goldfarb-Tarrant. 2025. Safer or Luckier? LLMs as Safety Evaluators Are Not Robust to Artifacts. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*.

[3] Emily Dinan et al. 2019. Build it Break it Fix it for Dialogue Safety: Robustness from Adversarial Human Attack. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. 4537–4546.

[4] Samuel Gehman et al. 2020. RealToxicityPrompts: Evaluating Neural Toxic Degeneration in Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 3356–3369.

[5] Kai Greshake et al. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173

[6] Jiacheng Gu et al. 2024. A Survey on LLM-as-a-Judge. arXiv:2411.15594

[7] Tao Huang. 2025. Content Moderation by LLM: From Accuracy to Legitimacy. *Artificial Intelligence Review* (2025).

[8] Hakan Inan et al. 2023. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. arXiv:2312.06674

[9] Seungone Kim et al. 2024. Prometheus: Inducing Fine-grained Evaluation Capability in Language Models. In *The Twelfth International Conference on Learning Representations*.

[10] Tianle Li, Xingjun Zheng, and Xuanjing Huang. 2024. Open the Pandora's Box of LLMs: Jailbreaking LLMs through Representation Engineering. arXiv:2401.06824

[11] Yinhong Liu et al. 2023. G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2511–2522.

[12] Mantas Mazeika et al. 2024. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. In *Proceedings of the 41st International Conference on Machine Learning*.

[13] Long Ouyang et al. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, Vol. 35. 27730–27744.

[14] Vyas Raina, Adian Liusie, and Mark Gales. 2024. Is LLM-as-a-Judge Robust? Investigating Universal Adversarial Attacks on Zero-shot LLM Assessment. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 6539–6563.

[15] Rickard Stureborg, Dimitris Alikaniotis, and Yoshi Suhara. 2024. Large Language Models are Inconsistent and Biased Evaluators. arXiv:2405.01724

[16] Tong Tong, Fei Wang, Zhiyuan Zhao, and Mingjie Chen. 2025. BadJudge: Backdoor Vulnerabilities of LLM-as-a-Judge. arXiv:2503.00596

[17] Peiqi Wang et al. 2024. Large Language Models are not Fair Evaluators. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 9440–9450.

[18] Yidong Wang et al. 2024. PandaLM: An Automatic Evaluation Benchmark for LLM Instruction Tuning Optimization. In *The Twelfth International Conference on Learning Representations*.

[19] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. 2023. Jailbroken: How Does LLM Safety Training Fail? arXiv:2307.02483

[20] Yuncong Xie, Mingyang Fang, Renwang Pi, and Neil Gong. 2024. GradSafe: Detecting Jailbreak Prompts for LLMs via Safety-Critical Gradient Analysis. arXiv:2402.13494

[21] Yu Yan, Sheng Sun, Zenghao Duan, Teli Liu, Min Liu, Zhiyi Yin, LeiJingyu Lei-Jingyu, and Qi Li. 2025. From benign import toxic: Jailbreaking the language model via adversarial metaphors. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 4785–4817.

[22] Yu Yan, Sheng Sun, Zixiang Tang, Teli Liu, and Min Liu. 2025. Collaborative stance detection via small-large language model consistency verification. In *International Conference on Database Systems for Advanced Applications*. Springer, 3–20.

[23] Yu Yan, Sheng Sun, Zhe Wang, Yijun Lin, Zenghao Duan, Min Liu, Jianping Zhang, et al. 2025. Confusion is the final barrier: Rethinking jailbreak evaluation and investigating the real misuse threat of LLMs. *arXiv preprint arXiv:2508.16347* (2025).

[24] Haoran Yang et al. 2025. Any Large Language Model Can Be a Reliable Judge: Debiasing with a Reasoning-based Bias Detector. arXiv:2505.17100

[25] Lianmin Zheng et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, Vol. 36.

[26] Lianghui Zhu, Xinggang Wang, and Xinlong Wang. 2023. JudgeLM: Fine-tuned Large Language Models are Scalable Judges. arXiv:2310.17631

[27] Andy Zou et al. 2023. Universal and Transferable Adversarial Attacks on Aligned Language Models. arXiv:2307.15043